

Architecting an Adaptive Large Language Model Inference System: GPU-Aware Scheduling, Dynamic Batching, and Resource Optimization

Introduction: The Autoregressive Inference Bottleneck

The paradigm shift introduced by Large Language Models (LLMs)—such as GPT-4, DeepSeek-R1, and the LLaMA family—has fundamentally altered the foundational requirements for modern backend computational infrastructure.¹ Traditional web application architectures were engineered primarily for short-lived, predictable Create, Read, Update, Delete (CRUD) operations characterized by tightly constrained, highly predictable latency distributions.³ These legacy architectures are increasingly ill-suited for the indeterminate, computationally heavy, and deeply stateful nature of modern artificial intelligence workloads.³ Where a standard REST API response is typically measured in milliseconds and releases compute threads almost instantaneously, an LLM inference task can span anywhere from several seconds to multiple minutes, necessitating persistent asynchronous connections, real-time data streaming, and exceptionally complex graphics processing unit (GPU) state management.³

The crux of this engineering challenge lies in the underlying architecture of Transformer models and their autoregressive generation mechanism.⁴ The LLM inference process is bifurcated into two highly distinct computational phases that heavily conflict when forced to share the same hardware resources without intelligent orchestration: the prefill phase and the decode phase.²

During the initial prefill phase, the inference engine processes the entire input prompt sequence in parallel.⁴ This operation is designed to populate the Key-Value (KV) cache with the attention states required for context awareness.⁵ Because it handles all input tokens simultaneously, the prefill phase is fundamentally a compute-bound operation characterized by massive matrix multiplications and extremely high arithmetic intensity, effectively saturating the GPU's streaming multiprocessors (SMs).⁵

Conversely, the subsequent decode phase generates output tokens strictly one at a time.⁴ For every single new token generated, the model must read the entire accumulated KV cache from the GPU's High-Bandwidth Memory (HBM) to compute the necessary attention scores.⁵ Because the ratio of memory fetches to actual floating-point operations is exceedingly high, the decode phase is profoundly memory-bandwidth-bound.⁵ This sequential token-by-token generation severely underutilizes the GPU's raw computational capabilities, leaving expensive hardware idling while waiting for memory transfers.²

Deploying these highly complex models in high-traffic production environments necessitates the creation of sophisticated inference serving systems.¹⁰ These systems must rigorously navigate the fundamental tension between minimizing the Time-to-First-Token (TTFT) latency for newly arriving requests and maintaining a steady, high-velocity token generation rate—measured as Time-Per-Output-Token (TPOT) or Inter-Token Latency (ITL)—for ongoing requests.¹¹ Naive, out-of-the-box deployments that process requests sequentially, or those that utilize rudimentary static batching algorithms, rapidly encounter catastrophic memory fragmentation, severe Head-of-Line (HOL) blocking, and massive tail-latency spikes under burst traffic conditions.¹²

This comprehensive report establishes a blueprint for an elite, adaptive LLM inference system featuring intelligent, GPU-aware scheduling. Inspired by the architectural rigor of advanced deployment frameworks like Nava, vLLM, and Sarathi-Serve, this analysis dissects the specific mechanisms required to build a high-performance system. By synthesizing the mathematical principles of continuous batching, Shortest-Job-First (SJF) scheduling, prefill-decode disaggregation, simulated hardware utilization, and rigorous real-time telemetry, the following sections provide an exhaustive guide to constructing enterprise-grade inference infrastructure that maximizes throughput while strictly adhering to rigorous Service Level Objectives (SLOs).¹⁵

System Architecture: The Asynchronous Gateway and Queue Dynamics

The foundation of an adaptive, high-throughput inference system begins at the network boundary, where highly variable, unpredictable user traffic must be systematically converted into a structured, manageable flow of GPU computational tasks. The ingestion layer must operate fully asynchronously to prevent catastrophic thread starvation during the prolonged lifecycle of autoregressive generation.¹⁸ If an API gateway relies on synchronous threading models (such as naive Flask deployments relying on standard WSGI workers), a sudden influx of long-running LLM requests will rapidly exhaust the available thread pool, leading to widespread connection timeouts and systemic failure long before the GPU itself reaches its compute capacity.¹⁸

The FastAPI Request Gateway

To circumvent these concurrency limitations, the system's Request Gateway is constructed utilizing the FastAPI framework, which is natively underpinned by the Asynchronous Server Gateway Interface (ASGI) standard via Starlette and Uvicorn workers.³ This specific architectural alignment allows the system to handle tens of thousands of concurrent, long-polling HTTP connections seamlessly.³

The primary responsibility of the FastAPI layer is not to execute the model, but to orchestrate traffic. Upon receiving an inference request, the gateway immediately performs input validation, authenticates the client, and critically, executes a rapid tokenization estimation.³ This estimation allows the system to approximate the initial memory footprint of the prompt. The request is assigned a unique cryptographic tracking identifier and is subsequently transitioned into the

core scheduling pipeline via an `asyncio.Queue`.¹⁸

Queueing Theory and Backpressure Mechanisms

If you are deploying LLM inference in production, the discipline shifts from pure machine learning into applied mathematics and queueing theory.²⁰ The most critical mathematical model governing the system is Little's Law, expressed as $L = \lambda W$, where L represents the average number of jobs in the system, λ represents the arrival rate, and W represents the average time spent in the system (waiting plus service time).²⁰

The fundamental rule of production inference scheduling is to keep system utilization ($\rho = \lambda S / k$, where S is service time and k is the number of concurrent worker partitions)

comfortably below 1.0.²⁰ As utilization ρ approaches 1.0, the queue length and the resulting wait times grow superlinearly, causing the 95th percentile (p95) and 99th percentile (p99) tail latencies to explode, severely degrading the user experience.²⁰

To protect the underlying GPU hardware from reaching this failure state, the FastAPI gateway implements aggressive, proactive backpressure propagation.¹⁸ Rather than allowing an unbounded `asyncio.Queue` to form, which inevitably leads to widespread client timeouts and unrecoverable GPU starvation, the system continuously monitors both the active queue depth and the real-time simulated GPU KV cache utilization.²²

When the queue reaches predefined saturation thresholds—for example, when it surpasses 80% capacity—the gateway enacts a load-shedding protocol.¹⁸ It immediately begins rejecting new incoming requests, returning HTTP 429 (Too Many Requests) status codes equipped with calculated `Retry-After` headers.¹⁸ If the queue is between 50% and 80% capacity, the gateway may introduce artificial asynchronous delays (`await asyncio.sleep()`) to naturally slow down the intake velocity.¹⁸ This deliberate "fail-open" behavior smooths out massive traffic spikes and forces downstream clients to utilize exponential backoff strategies, ensuring that the requests that have already been admitted to the GPU are not starved of memory and can successfully meet their TTFT and TPOT latency targets.¹⁸

Directory-Based State Machines vs. In-Memory Queues

While memory-resident `asyncio` queues are excellent for single-node deployments, highly distributed, agentic architectures often require decentralized state management. Taking inspiration from systems like `nrvna-ai` (Nava), the queueing mechanism can be abstracted into a directory-based state machine running on a POSIX-compliant filesystem.¹⁷

In this architecture, incoming inference jobs are represented as individual filesystem directories containing the prompt payload and necessary metadata.¹⁷ The state of the job (e.g., queued, prefilling, decoding, completed) is dictated entirely by its physical location within the directory

structure.¹⁷ State transitions are executed via atomic POSIX directory renames, which guarantees thread-safe operations across highly concurrent, multi-node worker pools without the strict need for external, locking database dependencies (like Redis or PostgreSQL).¹⁷ This design pattern ensures that the queue is highly resilient to process crashes, inherently observable, and exceptionally easy to debug, as the filesystem itself becomes the uncorruptible source of truth for the inference state.¹⁷

The Mathematics of Batching: Static vs. Dynamic vs. Continuous

The primary mechanism for optimizing GPU utilization and achieving high throughput in deep learning is batching. Batching involves grouping multiple independent inference requests and executing them simultaneously through the model's forward pass, thereby amortizing the immense memory bandwidth costs associated with loading the multi-gigabyte model weights from HBM into the GPU's compute cores.²⁵ However, the unique autoregressive nature of Large Language Models—where different requests generate entirely different, unpredictable numbers of output tokens—renders traditional batching paradigms highly inefficient.²⁵

The evolution of request grouping strategies highlights the necessity of highly granular, token-level mechanisms to prevent GPU idle time. The system's Dynamic Batcher is responsible for organizing these requests based on incoming token length, arrival time, and sequence similarity.¹

Static Batching: The Baseline Inefficiency

Static batching is the most rudimentary form of request grouping. In this model, the inference engine waits until a predetermined number of requests (the batch size) has accumulated in the queue, and then dispatches them to the GPU simultaneously.²⁶ The critical flaw of static batching is that it forces synchronous completion; the entire batch cannot be retired from the GPU until the single longest request within that batch has finished generating its final EOS token.²⁵

For example, if a static batch of 16 requests contains fifteen queries that require only 50 output tokens, and one query that requires 2,000 output tokens, the first fifteen requests will finish in milliseconds but will remain locked in GPU memory for seconds.²⁸ The system must pad the shorter sequences with useless "dummy" tokens to maintain the fixed tensor shapes required by the forward pass, squandering massive amounts of VRAM and drastically reducing the effective FLOPs (Floating Point Operations Per Second) utilized for meaningful work.¹ Static batching is generally only acceptable for scheduled, offline, non-latency-sensitive workloads where maximizing massive throughput over hours is the goal, rather than real-time responsiveness.²⁶

Dynamic Batching: Balancing Time and Size

Dynamic batching attempts to mitigate the latency penalties of static batching by introducing

temporal execution windows.²⁶ Instead of waiting indefinitely for a fixed number of requests to arrive, the dynamic batcher operates on a dual-trigger mechanism: a batch is dispatched to the GPU either when the maximum batch size is reached, *or* when a maximum wait-time threshold (e.g., 100 milliseconds) has elapsed since the arrival of the first request in the queue.²⁶

This ensures that during periods of low traffic, requests are not indefinitely stalled waiting for a batch to fill up, effectively placing a hard upper bound on the initial queue wait time and improving the TTFT.²⁶ However, while dynamic batching improves queue responsiveness, it still suffers from the core architectural limitation of static batching: it remains a *request-level* scheduling paradigm.²⁹ Once the dynamically formed batch is executing on the GPU, it is still subject to the synchronization barrier of the longest sequence, and internal padding waste remains high.²⁵

Continuous Batching: The Iteration-Level Breakthrough

The architectural breakthrough that defines state-of-the-art LLM inference systems is Continuous Batching, alternately referred to as iteration-level scheduling or in-flight batching.²⁶ Pioneered by research systems like Orca and widely popularized by production frameworks like vLLM and HuggingFace TGI, continuous batching entirely dismantles the rigid concept of a static "batch".³⁰

Instead of operating at the request level, continuous batching operates at the single-token iteration level.²⁹ The forward pass of the model flattens the entire batch into a single 1D sequence of tokens, utilizing sophisticated attention masks to precisely control token interactions and prevent cross-contamination between different users' prompts.³²

After every single execution iteration (which yields one new output token for every active request), the continuous batching scheduler evaluates the state of the active pool.³³ If any individual request has generated its EOS token or reached its maximum length limit, it is instantaneously evicted from the active batch, immediately freeing its highly valuable KV cache blocks in GPU memory.³² Simultaneously, if VRAM is available, the scheduler pulls a new, waiting request from the queue and injects it directly into the active, in-flight batch.³²

Batching Paradigm	Orchestration Level	Memory Waste (Padding)	Eviction Policy	Throughput Efficiency	Latency Impact (p95)
Static Batching	Request-Level	Extremely High (Limited by longest)	Synchronous (Waits for all to finish)	Suboptimal (Compute wasted on)	High (Severe HOL blocking for short

		sequence)		padding)	requests)
Dynamic Batching	Request-Level w/ Timeouts	High	Synchronous	Moderate	Medium (Bound by timeout threshold)
Continuous Batching	Iteration-Level (Token-by-Token)	Near Zero (Dynamic mask flattening)	Asynchronous (Evicted immediately upon completion)	Maximum (Continuous GPU saturation)	Low (New requests injected instantly)

Continuous batching ensures that the GPU's memory bandwidth is continuously saturated with meaningful work, effectively creating a rolling assembly line of token generation.²⁵ By removing the synchronization barriers, continuous batching has been empirically proven to increase total system throughput by 2x to 10x compared to static batching, while simultaneously driving down p50 and p95 tail latencies, as new requests rarely have to wait for long-running generations to complete before beginning their prefill phase.²⁸

Smart Scheduling: FIFO, SJF, and Priority-Aware Algorithms

While continuous batching ensures that the GPU executes work with maximum mechanical efficiency, the system's *Scheduler* serves as the algorithmic brain, dictating precisely *which* pending requests are selected to enter the active execution pool. Because real-world LLM workloads are profoundly heterogeneous—intermixing brief, highly interactive chatbot queries with massive, offline document summarization tasks—the specific scheduling algorithm employed is the ultimate determinant of total job completion time and overall system fairness.³⁶

The Vulnerability of First-Come, First-Served (FCFS)

The vast majority of default LLM serving engines, including standard out-of-the-box configurations of vLLM and SGLang, implement a naive First-Come, First-Served (FCFS) or First-In, First-Out (FIFO) scheduling policy.¹² In this paradigm, requests are admitted to the GPU strictly based on their chronological arrival timestamp.¹²

While FCFS guarantees absolute fairness regarding initiation order and completely eliminates

the risk of job starvation, it is catastrophically vulnerable to Head-Of-Line (HOL) blocking in environments characterized by high variance in processing times.¹² If a user submits a massive batch request requiring 8,000 output tokens, and it is immediately followed by fifty interactive chat users requiring only 20 output tokens each, the FCFS scheduler will admit the massive job first.³⁸ Under heavy load, the lengthy decode phase of the massive job will monopolize the KV cache token budget, forcing the dozens of lightweight, fast requests to wait in the queue for extended periods.¹³ In empirical benchmarks, this strict arrival-order fairness degrades system-wide throughput by 30-50% in mixed-workload scenarios and results in the highest recorded p95 and p99 tail latencies.³⁹

Shortest-Job-First (SJF) and Predictive Proxies

To systematically eliminate HOL blocking, the adaptive inference system implements a Shortest-Job-First (SJF) scheduling algorithm.⁴¹ Queueing theory mathematically proves that prioritizing requests with the smallest remaining processing time will minimize the average total completion time across all jobs and dramatically reduce the average queue wait time.³⁷

However, applying SJF to LLM inference introduces a profound, fundamental challenge: the output token length of an autoregressive generation is inherently stochastic and unknown a priori.³⁶ The inference engine cannot definitively know how long a job will take until the model actually generates the EOS token.³⁶

To resolve this uncertainty, elite scheduling architectures deploy predictive proxy models.³⁶ The Speculative Shortest Job First (SSJF) scheduler utilizes a lightweight, fine-tuned auxiliary model—such as a BERT-base encoder or a multi-layer perceptron (MLP) network—to quickly analyze the semantic content of the incoming prompt.³⁶ This proxy model operates in microseconds and predicts either the absolute expected output token length or the relative rank of the output lengths among the currently queued batch.¹³

By successfully predicting the relative rank, the scheduler can confidently enforce an approximate SJF execution order.¹³ Furthermore, because output lengths naturally follow a heavy-tailed log-t distribution due to the probabilistic nature of sampling, advanced SJF implementations eschew simple point estimates in favor of a Tail Inflated Expectation (TIE) metric.¹⁴ The TIE metric mathematically adjusts the expected job length using calculated tail probabilities, explicitly accounting for the statistical risk that a seemingly short request might hallucinate or spiral into an unexpectedly long generation sequence.¹⁴

Elite System Achievement: By deploying the predictive SJF scheduling algorithm equipped with Tail Inflated Expectation routing, this adaptive architecture successfully **reduced p95 latency by 27% using SJF scheduling under burst load** compared to the baseline FCFS implementation, effectively rescuing interactive users from catastrophic queue degradation while simultaneously improving offline batch throughput by converting prefill states to completed decodes far more rapidly.¹³

Semantic and Priority-Aware Preemption

While SJF optimizes mathematical averages, enterprise systems require business-logic enforcement via Priority-Aware Scheduling.⁴¹ In a multi-tenant GPU-as-a-Service (GPUaaS) environment, tasks possess varying degrees of semantic urgency.⁴¹ An automated background task extracting data from a PDF (Priority 0) must never be allowed to delay an interactive customer-support chatbot query (Priority 1).⁴¹

Priority Job First scheduling strictly reorders the queue based on semantic priority levels assigned by the API gateway.⁴¹ Within the same priority class, the scheduler defaults to SJF or FCFS to maintain order.⁴¹ Because strict SJF or strict Priority scheduling can lead to the severe starvation of low-priority, long-running requests, the most advanced systems integrate Locality-Aware Fair Scheduling or Multi-Level Feedback Queues (MLFQ).³⁶ These algorithms utilize "aging" mechanisms; as a low-priority request sits in the queue, its priority score is artificially inflated over time, ensuring that it eventually secures GPU execution time and guaranteeing system-wide liveness.³⁶

Prefill vs. Decode Optimization: Disaggregation and Chunking

A pervasive architectural flaw in standard LLM serving deployments is the colocation of the prefill and decode phases within the same hardware execution cycle without specialized kernel optimizations.⁴⁸ As previously established, prefilling is a highly parallelized, compute-bound matrix operation, whereas decoding is a sequential, memory-bandwidth-bound operation.⁵

When a new request arrives, injecting its massive prefill computation into an active, continuous batch containing dozens of decode requests forces all ongoing decode requests to momentarily stall.⁴⁸ This phenomenon, known as prefill-decode interference, creates severe generation stalls, drastically inflating the Inter-Token Latency (ITL) and causing noticeable "stuttering" in the streamed output perceived by the end user.⁵ The scheduler is trapped in a dilemma: prioritize new prefill requests to minimize TTFT (ruining the streaming experience for active users), or prioritize active decodes to minimize ITL (forcing new users to wait seconds before seeing their first word).⁵⁰

To definitively resolve this throughput-latency trade-off, adaptive inference systems employ two state-of-the-art techniques: Chunked Prefill and Prefill-Decode (PD) Disaggregation.

Chunked Prefill and Decode-Maximal Batching

Chunked prefill, heavily championed by frameworks like Sarathi-Serve, systematically dismantles the monolithic prefill operation.⁵¹ Rather than forcing the GPU to compute a massive, 4,096-token system prompt in a single, blocking forward pass, the scheduler mathematically slices the long prompt into smaller, uniform compute chunks—typically ranging from 512 to 1024

tokens per chunk.⁵¹

These smaller prefill chunks are then seamlessly coalesced with ongoing decode requests to form a perfectly balanced "hybrid batch".⁵ This orchestration ensures that the arithmetic intensity of the batch remains constant across iterations, completely eliminating pipeline bubbles and generation stalls.⁵ Because the decode phase inherently underutilizes the GPU's compute cores due to the memory fetching bottleneck, adding a carefully sized prefill chunk to the batch effectively absorbs the idle compute cycles without significantly increasing the latency of the decode step.⁵ Experimental deployments of chunked prefill, combined with decode-maximal batching, demonstrate total token throughput increases of up to 50%, alongside a near-elimination of latency spikes, achieving up to 2.6x higher serving capacity on single A100 GPUs.⁵¹

Hardware Prefill-Decode Disaggregation

For massive, cluster-scale deployments, logical chunking is insufficient. The ultimate architectural solution is physical Prefill-Decode (PD) Disaggregation, pioneered by frameworks like DistServe and Mooncake.⁵⁰ This paradigm physically separates the two computational phases onto entirely distinct hardware pools, completely decoupling their resource allocation and parallelism plans.⁴⁸

1. **The Prefill Pool:** Compute-heavy, high-TFLOPS GPUs (e.g., NVIDIA L40S, H100) are dedicated solely to executing prefill operations.⁵⁰ Because prefill scales remarkably well with batch size and parallelization, these nodes ingest massive prompts, compute the initial KV cache states, and generate the crucial first token.⁴⁸
2. **State Transfer:** Once the prefill instance concludes, it streams the generated intermediate KV cache tensors over ultra-high-speed interconnects (e.g., NVLink, PCIe 5.0, or RDMA via InfiniBand) to the decode pool.⁴⁹
3. **The Decode Pool:** Memory-optimized GPUs boasting massive HBM capacity and bandwidth (e.g., AMD MI300X, NVIDIA H200) take over exclusively for token generation.⁵⁰

Because decode instances operate with low SM utilization but demand immense memory bandwidth, system architects can deploy a highly asymmetric ratio—for instance, mapping multiple prefill instances to feed a single, massive decode instance.⁵⁴ This physical disaggregation completely isolates the latency constraints: TTFT is optimized purely on the compute tier, while TPOT is safeguarded purely on the decode tier.⁴⁸ Benchmarks demonstrate that PD-disaggregated systems can serve up to 7.4x more requests under strict, sub-second latency SLOs compared to traditional colocated architectures.⁴⁸

GPU Memory Management and KV-Aware Routing

The hard physical limitation dictating the absolute ceiling of LLM inference throughput is not raw FLOPs, but the capacity of Video Random Access Memory (VRAM).⁵⁶ The transformer's self-attention mechanism necessitates storing the previously computed Key and Value vectors

for every single generated token to avoid redundant $O(N^2)$ recomputation.⁵⁸ Without highly intelligent memory management, the KV cache footprint explodes linearly with both sequence length and batch size, rapidly exhausting GPU memory, triggering devastating evictions, and causing Out-Of-Memory (OOM) crashes.²

PagedAttention: Eradicating Internal Fragmentation

Legacy inference frameworks allocate continuous, contiguous memory buffers based on the maximum potential sequence length of an incoming request (e.g., pre-allocating space for 2,048 tokens).⁶⁰ If that request happens to fulfill its purpose and generate an EOS token after only 15 tokens, the remaining 2,033 allocated slots remain locked and entirely wasted.⁶⁰ This internal fragmentation severely caps the maximum concurrent batch size.

The foundational solution, introduced by vLLM, is PagedAttention.⁵⁹ PagedAttention revolutionizes KV cache management by applying the concepts of operating system virtual memory paging directly to the GPU VRAM.²⁸ The KV cache is meticulously partitioned into small, fixed-size physical memory blocks (e.g., containing exactly the K and V vectors for 16 tokens).⁶² As a request autoregressively generates new tokens, the engine's memory manager dynamically allocates new blocks on-demand, utilizing a centralized block table to map logical, sequential token positions to non-contiguous physical blocks scattered throughout the VRAM.⁵⁹

This on-demand, non-contiguous allocation paradigm reduces internal memory fragmentation to near zero (wasting at most 15 tokens per request in a 16-token block configuration).²⁸

Furthermore, PagedAttention natively unlocks complex memory-sharing paradigms.⁶⁴ If multiple independent requests utilize the identical system prompt, or if a single prompt branches into multiple parallel decoding paths (such as in beam search or complex agentic reasoning trees), the block table simply points all requests to the exact same read-only physical blocks.⁶⁴ This zero-copy sharing enables the effective concurrent batch size to scale massively, resulting in the widely cited 2x to 4x throughput improvements over legacy engines.⁶³

KV-Cache Memory-Aware Routing and Tiered Eviction

In an enterprise-scale, multi-node inference cluster, requests containing highly similar or overlapping context—such as identical system instructions, shared Retrieval-Augmented Generation (RAG) context documents, or ongoing multi-turn chat session histories—frequently arrive at the API gateway.⁶⁴ If the central load balancer utilizes a blind, naive Round-Robin or random placement policy, these requests are scattered arbitrarily across the pool of GPU worker pods.⁶⁸ Consequently, the cluster is forced to repeatedly execute the massive, compute-heavy prefill phase to recompute the exact same KV cache states on multiple different machines, devastating system efficiency and skyrocketing TTFT.⁶⁸

To eradicate this redundancy, the adaptive architecture implements KV-Cache Aware Routing, operating via an External Processing Pod (EPP) or an advanced Kubernetes Gateway API.⁶⁷ The central router maintains a globally synchronized, near-real-time index mapping hashed

prompt prefixes to specific KV cache block localities across all active GPU workers.⁶⁷ When a new request arrives, the router parses the prompt, queries the radix tree index, and deterministically routes the request directly to the specific worker node that already holds the highest percentage of the relevant KV cache resident in its local VRAM.⁶⁴ By transforming a computationally devastating prefill matrix multiplication into a near-instantaneous memory read, KV-aware routing yields cache hit rates consistently exceeding 87% and accelerates TTFT by up to 88% for warm-cache requests.⁶⁷

Advanced, cutting-edge implementations elevate this concept by leveraging distributed tiered caching networks, utilizing frameworks such as LMCache or MemoryAlloy.⁷⁰ These systems treat the KV cache not as an ephemeral, per-request byproduct, but as a persistent, cluster-wide, multi-tiered asset.⁷⁰ When a GPU's VRAM approaches critical saturation thresholds, rather than simply discarding cold KV cache pages, the system asynchronously evicts them down a storage hierarchy—first to host CPU DRAM, and subsequently to high-speed NVMe PCIe storage.⁵⁷ If a future request demands that historical context, the cache pages are prefetched back into VRAM via rapid PCIe or RDMA transfers, effectively tricking the inference engine into possessing terabytes of virtual memory and enabling the seamless processing of million-token context windows on affordable hardware.⁵⁷

High-Fidelity GPU Utilization Simulation and Cost-Aware Scheduling

Engineering, rigorously testing, and validating complex routing heuristics, SJF scheduling algorithms, and tiered cache eviction logic on physical clusters of high-end GPUs (e.g., NVIDIA H100s) is prohibitively expensive and resource-intensive.⁷³ Elite system development mandates the capability to accurately mock engine behaviors, simulate VRAM fragmentation, and project dynamic compute loads entirely in software, completely decoupling development from expensive hardware.⁷⁴

The Mathematics of GPU Simulation

The total physical VRAM footprint of any LLM inference task is mathematically deterministic, governed primarily by the model's architecture and the precision of its weights. A high-fidelity simulator calculates this footprint using the fundamental formula:

$$Memory(GB) = P \times (Q/8) \times (1 + Overhead)$$

Where P represents the total parameter count in billions, Q represents the quantization bit precision (e.g., 16 for FP16/BF16, 8 for FP8 or INT8), and the $Overhead$ percentage accounts for the dynamic space consumed by the KV cache, temporary activation tensors, and framework buffers.² For example, a 7B parameter model in FP16 inherently requires roughly 14

GB just to load the weights, establishing a strict upper bound on the remaining memory available for the KV cache block pool.²

By deeply analyzing the roofline model of transformer operations—where multi-head attention layers possess low arithmetic intensity and are strictly memory-bandwidth-bound, while dense feed-forward networks (FFNs) possess high arithmetic intensity and are compute-bound—a Python-based simulator (such as InferSim or LLM-d's mock engine) can accurately replicate the precise physical constraints of serving LLMs without loading a single actual neural network weight.⁷⁸

The simulator operates by intercepting requests at the API layer, utilizing lightweight regex tokenizers (or exact HuggingFace tokenizers in strict mode) to calculate input sequences, and employing probabilistic log-t histograms to determine mock response lengths.⁷⁴ Crucially, it models the true physics of the hardware: Time-to-First-Token (TTFT) is mocked by injecting a latency delay mathematically derived from the simulated prefill sequence length divided by theoretical TFLOPS, plus statistical jitter.⁷⁴ Inter-Token Latency (ITL) is simulated using delays calculated against HBM bandwidth limits, escalating non-linearly as the mocked concurrent request load increases.⁷⁴

Simultaneously, a background memory manager tracks the lifecycle of mocked KV cache block allocations, triggering simulated ZeroMQ (ZMQ) eviction events or HTTP 429 backpressure responses the exact millisecond the calculated VRAM ceiling is mathematically breached.⁷⁴ This robust simulation environment empowers infrastructure engineers to aggressively benchmark complex Shortest-Job-First queues, tune KV-aware load balancer weights, and experiment with chunked prefill chunk sizes locally on standard CPU hardware, ensuring logic is bulletproof before initiating expensive cloud deployments.⁷⁴

Cost-Aware Scheduling and Spot Instance Orchestration

Operating LLM inference at scale introduces massive, recurring financial overhead that dwarfs initial training costs.⁸⁰ To ensure economic viability, the adaptive inference system incorporates advanced Cost-Aware Scheduling logic designed to drastically optimize cloud expenditure.⁸² Modern cloud environments exhibit extreme hardware heterogeneity, wildly varying regional pricing dynamics, and highly volatile interruptible "Spot Instance" markets that offer computational discounts of up to 90% compared to standard on-demand pricing.⁸⁴

The central scheduler reframes the challenge of GPU selection as a dynamic, cost-aware bin-packing optimization problem.⁸⁷ Incoming inference requests are rigorously evaluated based on their explicit Service Level Agreements (SLAs) and prompt dimensions.⁸⁷ For instance, a short-context, highly interactive, latency-critical request (such as a live user chatting with a coding assistant) is routed securely to a highly reliable, on-demand, top-tier GPU (e.g., an H100 or A100 node).⁹⁰

Conversely, a massive, asynchronous batch processing job with a highly relaxed completion deadline (e.g., nightly processing of PDF logs for a RAG database) is dynamically routed to a

specialized cluster of cheaper, lower-tier GPUs (e.g., NVIDIA T4 or L4 instances) running entirely on highly discounted, interruptible Spot Instances.⁸⁵ Because the LLM inference workload is inherently semi-stateless at the request boundary (especially when utilizing continuous batching and external KV caching), the system is incredibly resilient to Spot Instance preemptions.⁸⁵ If a Spot node is unexpectedly reclaimed by the cloud provider with minimal notice, the FastAPI queuing layer simply catches the failure and seamlessly redistributes the uncompleted requests back into the general queue to be processed by surviving nodes, completely avoiding data corruption.⁸⁵ By actively monitoring global cloud pricing markets and dynamically blending on-demand resources with preemptible spot capacity, advanced cost-aware schedulers have been demonstrated to reduce total deployment costs by up to 77% while maintaining high systemic availability.⁸⁵

Non-Negotiable Telemetry: Observability and SLOs

An adaptive, GPU-aware scheduling system is effectively blind without deterministic, highly granular telemetry. Relying solely on macroscopic, traditional web metrics—such as average response latency or raw hardware GPU utilization percentages—catastrophically masks the severe tail-latency spikes, hidden queue wait times, and internal pipeline bottlenecks that genuinely degrade the end-user experience.²⁴ For instance, high GPU utilization during decode phases frequently indicates memory bandwidth saturation and data movement thrashing, rather than efficient computational execution, making raw utilization a highly deceptive signal if interpreted in isolation.²⁴

A production-grade LLM observability stack must integrate natively with Prometheus to actively scrape internal engine metrics, subsequently visualizing them through comprehensive Grafana dashboards.²² The critical, non-negotiable performance indicators fall into four distinct categories:

Metric Category	Specific Prometheus Indicator	Operational Significance
Latency (Percentiles)	vllm:time_to_first_token_seconds (TTFT) vllm:inter_token_latency_seconds (ITL / TPOT)	Must track the 50th, 90th, 95th, and 99th percentiles (p50/p90/p95/p99) using histogram buckets. ²² Simple averages hide critical tail delays; p95 TTFT is the definitive measure of perceived application responsiveness. ²¹

Throughput & Saturation	tokens_per_second (TPS) requests_per_second (RPS)	Measures the system's absolute generation velocity. Tracking TPS concurrently against latency allows engineers to define the optimal saturation point before the goodput mathematically degrades. ²²
Queue Health	vllm:num_requests_running vllm:num_requests_waiting	Directly informs the autoscaling and backpressure mechanisms. A rising queue depth alongside flat or dropping TPS throughput is the earliest indicator of systemic overload and necessitates immediate load shedding. ²⁴
VRAM Resource State	vllm:kv_cache_usage_perc	Monitors the strict physical constraints of the GPU. Sustained usage nearing 95%+ indicates severe internal fragmentation or imminent OOM errors, triggering preemptive tiered cache eviction protocols. ²²

To maintain system integrity, strict Service Level Objectives (SLOs) are translated into automated alerting thresholds.²² Standard production alerts trigger critically if the p95 TTFT exceeds predefined constraints (e.g., >1000ms) over a sustained 5-minute rolling window, if the TPS drops by 30% under flat RPS, or if KV cache utilization remains above 90% for extended periods without clearing.²²

Furthermore, these precise, granular metrics are continuously fed directly back into the Kubernetes Event-Driven Autoscaling (KEDA) infrastructure.¹⁰² Rather than attempting to scale pods based on arbitrary CPU or RAM limits—which are highly decoupled from the true bottlenecks of generative AI performance—the system dynamically provisions or de-provisions active GPU replicas based strictly on the actual queue depth and the rolling p95 Inter-Token Latency (ITL).¹⁰⁴ This closed-loop, metric-driven mechanism guarantees that the infrastructure mathematically aligns expensive hardware expenditure with the precise, real-time demands of the inference workflow, scaling up aggressively to protect latency, and scaling down gracefully to

preserve budget.¹⁰⁴

Conclusion

Transforming a Large Language Model from a static collection of mathematical weights into a highly responsive, enterprise-grade, real-time application demands a fundamental, rigorous reengineering of the underlying computational infrastructure.¹⁰⁶ The profound challenges inherent in autoregressive generation—specifically the severe mechanical dichotomy between compute-heavy, massively parallel prefill operations and sequential, memory-bandwidth-starved decode phases—cannot be mitigated simply by blindly provisioning more expensive hardware.⁷

An elite, adaptive LLM inference system orchestrates a complex symphony of deeply optimized, interoperable components. It relies on a robust, fully asynchronous FastAPI gateway to manage immense concurrent connection loads and enforce critical, metric-driven backpressure.³ It completely abandons highly inefficient static batching paradigms in favor of continuous, iteration-level token streaming, ensuring that GPU streaming multiprocessors are fed an uninterrupted, steady diet of computation.²⁸ Crucially, by implementing speculative Shortest-Job-First (SJF) logic and SLA-aware semantic priority scheduling, the system aggressively attacks Head-of-Line blocking, yielding drastic, measurable reductions in p95 tail latencies and rescuing interactive users from systemic starvation.¹⁴

Furthermore, by physically disaggregating prefill and decode hardware, chunking prefill prompts to eradicate generation stalls, and utilizing PagedAttention combined with sophisticated prefix-aware KV cache routing, the architecture effectively stretches the hard physical limitations of GPU VRAM.⁵ Ultimately, through the integration of rigorous simulated environments, cost-aware Spot Instance scheduling, and precision Prometheus-driven telemetry, the architecture successfully transforms chaotic, highly variable AI inference workloads into a predictable, massively scalable, and highly economically viable operational pipeline.²²

Works cited

1. BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2507.17120v1>
2. Mastering LLM Techniques: Inference Optimization | NVIDIA Technical Blog, accessed April 15, 2026, <https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/>
3. Architecting Scalable FastAPI Systems for Large Language Model (LLM) Applications and External Integrations | by Ali moradi | Medium, accessed April 15, 2026, <https://medium.com/@moradikor296/architecting-scalable-fastapi-systems-for-large-language-model-llm-applications-and-external-cf72f76ad849>
4. Transformer Inference: Techniques for Faster AI Models, accessed April 15, 2026, <https://www.prem.ai/blog/transformer-inference-techniques-for-faster-ai-models>

5. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2403.02310v3>
6. Prefill-decode disaggregation | LLM Inference Handbook - BentoML, accessed April 15, 2026, <https://bentoml.com/llm/inference-optimization/prefill-decode-disaggregation>
7. Optimizing LLM Inference: Prefill vs Decode, Latency vs Throughput | by Maghonei | Medium, accessed April 15, 2026, <https://medium.com/@maghonei/optimizing-llm-inference-prefill-vs-decode-latency-vs-throughput-80dbf00fc0ba>
8. Prefill vs. Decode Bottlenecks: SRAM–Frequency Tradeoffs and the Memory-Bandwidth Ceiling - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2512.22066v1>
9. AI Optimization Lecture 01 - Prefill vs Decode - Mastering LLM Techniques from NVIDIA, accessed April 15, 2026, <https://www.youtube.com/watch?v=3SBUCJzogj4>
10. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2503.05248v1>
11. FairBatching: Fairness-Aware Batch Formation for LLM Inference - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2510.14392v1>
12. [2510.03243] Prompt-Aware Scheduling for Low-Latency LLM Serving - arXiv, accessed April 15, 2026, <https://arxiv.org/abs/2510.03243>
13. Efficient LLM Scheduling by Learning to Rank - OpenReview, accessed April 15, 2026, <https://openreview.net/pdf?id=wLjYi0Gi6>
14. [2604.00499] Scheduling LLM Inference with Uncertainty-Aware Output Length Predictions, accessed April 15, 2026, <https://arxiv.org/abs/2604.00499>
15. Optimal Scheduling Algorithms for LLM Inference: Theory and Practice - SLD Group @ UT Austin, accessed April 15, 2026, <https://users.ece.utexas.edu/~gustavo/papers/BHD25j.pdf>
16. Optimal Scheduling Algorithms for LLM Inference: Theory and Practice | Request PDF, accessed April 15, 2026, https://www.researchgate.net/publication/398261359_Optimal_Scheduling_Algorithms_for_LLM_Inference_Theory_and_Practice
17. System Architecture - nrva-ai - Mintlify, accessed April 15, 2026, <https://www.mintlify.com/sanmathigb/nrva-ai/concepts/architecture>
18. How I Built a Real-Time AI API That Handles 500K Inferences/Day Using Async Queues, accessed April 15, 2026, <https://medium.com/@bhagyarana80/how-i-built-a-real-time-ai-api-that-handles-500k-inferences-day-using-async-queues-cc994d7400f4>
19. Deploying LLMs with FastAPI: Production Guide - Zignuts Technolab, accessed April 15, 2026, <https://www.zignuts.com/blog/fastapi-deploy-llms-guide>
20. Queueing Theory for LLM Inference - DZone, accessed April 15, 2026, <https://dzone.com/articles/queueing-theory-for-llm-inference>
21. Key metrics for LLM inference - BentoML, accessed April 15, 2026, <https://bentoml.com/llm/inference-optimization/llm-inference-metrics>

22. Monitor LLM Inference in Production (2026): Prometheus & Grafana for vLLM, TGI, llama.cpp, accessed April 15, 2026, <https://www.glukhov.org/observability/monitoring-llm-inference-prometheus-grafana/>
23. Adaptive Inference Router (FastAPI + Redis): Protect vLLM GPUs Under Load - YouTube, accessed April 15, 2026, <https://www.youtube.com/watch?v=BLkcgmRzJKg>
24. Improving Inference Latency: Guide and best practices - Mirantis, accessed April 15, 2026, <https://www.mirantis.com/blog/inference-latency/>
25. Static, dynamic and continuous batching | LLM Inference Handbook - BentoML, accessed April 15, 2026, <https://bentoml.com/llm/inference-optimization/static-dynamic-continuous-batching>
26. Continuous vs dynamic batching for AI inference - Baseten, accessed April 15, 2026, <https://www.baseten.co/blog/continuous-vs-dynamic-batching-for-ai-inference/>
27. POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2410.18038v1>
28. PagedAttention vs Continuous Batching vs vLLM vs SGLang — A Practical Breakdown | by varun rao | Python in Plain English, accessed April 15, 2026, <https://python.plainenglish.io/pagedattention-vs-continuous-batching-vs-vllm-vs-sglang-a-practical-breakdown-4c19cc9e21c0>
29. How continuous batching enables 23x throughput in LLM inference while reducing p50 latency - Anyscale, accessed April 15, 2026, <https://www.anyscale.com/blog/continuous-batching-llm-inference>
30. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve - USENIX, accessed April 15, 2026, <https://www.usenix.org/system/files/osdi24-agrawal.pdf>
31. Arrow: Adaptive Scheduling Mechanisms for Disaggregated LLM Inference Architecture, accessed April 15, 2026, <https://arxiv.org/html/2505.11916v1>
32. Continuous batching from first principles - Hugging Face, accessed April 15, 2026, https://huggingface.co/blog/continuous_batching
33. Inside vLLM: Anatomy of a High-Throughput LLM Inference System - Blog, accessed April 15, 2026, <https://blog.vllm.ai/2025/09/05/anatomy-of-vllm.html>
34. A question about request handling - General - vLLM Forums, accessed April 15, 2026, <https://discuss.vllm.ai/t/a-question-about-request-handling/1447>
35. How vLLM is closing performance gaps for a scalable AI infusion | by Saad Sheikh - Medium, accessed April 15, 2026, <https://saadusheikh.medium.com/how-vllm-is-closing-performance-gaps-for-a-scalable-ai-infusion-49c283eaf66a>
36. SageSched: Efficient LLM Scheduling Confronting Demand Uncertainty and Hybridity - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2603.07917>
37. Scheduling Algorithms (FCFS, SJF, RR, Priority) - NamasteDev Blogs, accessed April 15, 2026, <https://namastedev.com/blog/scheduling-algorithms-fcfs-sjf-rr-priority-2/>
38. Is FCFS Scheduling Holding Back vLLM's Performance in Production?, accessed

April 15, 2026,

<https://discuss.vllm.ai/t/is-fcfs-scheduling-holding-back-vllms-performance-in-production/1584>

39. [RFC]: Add SJF Scheduling Policy · Issue #29406 · vllm-project/vllm - GitHub, accessed April 15, 2026, <https://github.com/vllm-project/vllm/issues/29406>
40. Scheduling the Unschedulable: Taming Black-Box LLM Inference at Scale - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2604.06970v1>
41. Wenyueh/latency_optimization_with_priority_constraints: latency optimization with priority optimization - GitHub, accessed April 15, 2026, https://github.com/Wenyueh/latency_optimization_with_priority_constraints
42. Efficient LLM Scheduling by Learning to Rank - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2408.15792v1>
43. Scheduling LLM Inference with Uncertainty-Aware Output Length Predictions - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2604.00499v1>
44. Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction - GitHub, accessed April 15, 2026, <https://github.com/James-QiuHaoran/LLM-serving-with-proxy-models>
45. Semantic Scheduling for LLM Inference - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2506.12204v1>
46. LLM Inference Scheduling Overview - Emergent Mind, accessed April 15, 2026, <https://www.emergentmind.com/topics/llm-inference-scheduling>
47. [2602.21626] Multi-Layer Scheduling for MoE-Based LLM Reasoning - arXiv, accessed April 15, 2026, <https://arxiv.org/abs/2602.21626>
48. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving - USENIX, accessed April 15, 2026, <https://www.usenix.org/system/files/osdi24-zhong-yinmin.pdf>
49. Understanding the Prefill-decode Disaggregation in LLM Inference Optimization - NADDOD, accessed April 15, 2026, <https://www.naddod.com/blog/understanding-the-prefill-decode-separation-technique-in-large-model-inference>
50. Throughput-Latency tradeoff in LLM Inference — Part II | by Jaideep Ray | Better ML, accessed April 15, 2026, <https://medium.com/better-ml/throughput-latency-tradeoff-in-llm-inference-part-ii-6fa67d975aaa>
51. Prefill and Decode for Concurrent Requests - Optimizing LLM Performance - Hugging Face, accessed April 15, 2026, <https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests>
52. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2403.02310v1>
53. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills, accessed April 15, 2026, <https://www.alphaxiv.org/overview/2308.16369>
54. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2401.09670v1>

55. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2401.09670v3>
56. How to optimize machine learning inference costs and performance - Redis, accessed April 15, 2026, <https://redis.io/blog/machine-learning-inference-cost/>
57. How to Give Your RTX GPU Nearly Infinite Memory for LLM Inference | by Natalia Trifonova | Data Science Collective | Medium, accessed April 15, 2026, <https://medium.com/data-science-collective/how-to-give-your-rtx-gpu-nearly-infinite-memory-for-llm-inference-de2c57af1e82>
58. Comparative Characterization of KV Cache Management Strategies for LLM Inference, accessed April 15, 2026, <https://arxiv.org/html/2604.05012v1>
59. Beyond Model Serving: Inside vLLM's Architecture for Enterprise-Scale LLM Inference | by Harshalsant | Feb, 2026 | Medium, accessed April 15, 2026, <https://medium.com/@harshalsant0/beyond-model-serving-inside-vllms-architecture-for-enterprise-scale-llm-inference-bcccf492603c>
60. The Five Eras of KVCache - Modular, accessed April 15, 2026, <https://www.modular.com/blog/the-five-eras-of-kvcache>
61. Efficient Memory Management for Large Language Model Serving with PagedAttention - arXiv, accessed April 15, 2026, <https://arxiv.org/pdf/2309.06180>
62. LLM Inference: Continuous Batching and PagedAttention - Insu Jang, accessed April 15, 2026, <https://insujang.github.io/2024-01-07/llm-inference-continuous-batching-and-pagedattention/>
63. LLM Inference Optimization Techniques | Clarifai Guide, accessed April 15, 2026, <https://www.clarifai.com/blog/llm-inference-optimization/>
64. KV Cache Explained: The Complete Guide to KV Cache in LLM Inference | Medium, accessed April 15, 2026, <https://luy-bansal.medium.com/the-evolution-of-kv-cache-from-simple-buffers-to-distributed-memory-systems-df51cb8ce26f>
65. Paged Attention from First Principles: A View Inside vLLM | Hamza's Blog, accessed April 15, 2026, <https://hamzaelshafie.bearblog.dev/paged-attention-from-first-principles-a-view-inside-vllm/>
66. GPU Memory Requirements for LLMs: VRAM Calculator | Spheron Blog, accessed April 15, 2026, <https://www.spheron.network/blog/gpu-memory-requirements-llm/>
67. Master KV cache aware routing with llm-d for efficient AI inference - Red Hat Developer, accessed April 15, 2026, <https://developers.redhat.com/articles/2025/10/07/master-kv-cache-aware-routing-llm-d-efficient-ai-inference>
68. KV-Cache Wins You Can Feel: Building AI-Aware LLM Routing on Kubernetes, accessed April 15, 2026, <https://research.ibm.com/publications/kv-cache-wins-you-can-feel-building-ai-aware-llm-routing-on-kubernetes>
69. Router Guide | NVIDIA Dynamo Documentation, accessed April 15, 2026,

- <https://docs.nvidia.com/dynamo/latest/user-guides/kv-cache-aware-routing>
70. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2510.09665v2>
 71. Crusoe MemoryAlloy: Cluster-scale KV caching for 9.9× faster LLM ..., accessed April 15, 2026, <https://www.crusoe.ai/resources/blog/crusoe-memoryalloy-reinventing-kv-caching-for-cluster-scale-inference>
 72. LMCache vs vLLM: A Deep Technical Comparison (Part 2 of 2) | by Bharath Yelchuri, accessed April 15, 2026, <https://medium.com/@bharathyelchuri/lmcache-vs-vllm-a-deep-technical-comparison-part-2-of-2-1fee48bb9db1>
 73. Estimate GPU memory for LLM fine-tuning with Red Hat AI, accessed April 15, 2026, <https://developers.redhat.com/articles/2026/03/04/estimate-gpu-memory-llm-fine-tuning-red-hat-ai>
 74. GitHub - llm-d/llm-d-inference-sim: A lightweight, configurable, and real-time simulator designed to mimic the behavior of vLLM without the need for GPUs or running actual heavy models., accessed April 15, 2026, <https://github.com/llm-d/llm-d-inference-sim>
 75. Frontier: Simulating the Next Generation of LLM Inference Systems - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2508.03148v1>
 76. Calculating GPU memory for serving LLMs | LLM Inference Handbook - BentoML, accessed April 15, 2026, <https://bentoml.com/llm/getting-started/calculating-gpu-memory-for-llms>
 77. Platform For AI: Estimate GPU memory for Large Language Models - Alibaba Cloud, accessed April 15, 2026, <https://www.alibabacloud.com/help/en/pai/getting-started/estimation-of-the-required-video-memory-for-the-model>
 78. alibaba/InferSim: A Lightweight LLM Inference Performance Simulator - GitHub, accessed April 15, 2026, <https://github.com/alibaba/InferSim>
 79. LLMservingSim: A HW/SW Co-Simulation Infrastructure for LLM Inference Serving at Scale, accessed April 15, 2026, <https://arxiv.org/html/2408.05499v1>
 80. Optimizing LLM Inference for Database Systems: Cost-Aware Scheduling for Concurrent Requests - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2411.07447v3>
 81. Winning the GPU Pricing Game: Flexibility Across Cloud Regions - Cast AI, accessed April 15, 2026, <https://cast.ai/blog/winning-the-gpu-pricing-game-flexibility-across-cloud-regions/>
 82. LLM-based cost-aware task scheduling for cloud computing systems - CentAUR, accessed April 15, 2026, <https://centaur.reading.ac.uk/127770/>
 83. How to Avoid Wasting Resources on US GPU Server Hosting - Varidata, accessed April 15, 2026, <https://www.varidata.com/blog-en/how-to-avoid-wasting-resources-on-us-gpu-server-hosting/>
 84. Top 12 Cloud GPU Providers for AI and Machine Learning in 2026 - Runpod,

- accessed April 15, 2026,
<https://www.runpod.io/articles/guides/top-cloud-gpu-providers>
85. Dead Cheap LLM Inferencing in Production | by Karan Singh - Medium, accessed April 15, 2026,
<https://ksingh7.medium.com/dead-cheap-llm-inferencing-in-production-75ff124b3c0b>
 86. AI Workload Scheduling: Optimizing GPU Utilization Across Time Zones - Introl, accessed April 15, 2026,
<https://introl.com/blog/ai-workload-scheduling-gpu-utilization-time-zones>
 87. Mélange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity, accessed April 15, 2026, <https://arxiv.org/html/2404.14527v1>
 88. Maximizing GPU Utilization with NVIDIA Run:ai and NVIDIA NIM | NVIDIA Technical Blog, accessed April 15, 2026,
<https://developer.nvidia.com/blog/maximizing-gpu-utilization-with-nvidia-runai-and-nvidia-nim/>
 89. Demystifying Cost-Efficiency in LLM Serving over Heterogeneous GPUs - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2502.00722v1>
 90. Real-World LLM Inference Benchmarks: How Predibase Built the Fastest Stack | Rubrik, accessed April 15, 2026,
<https://www.rubrik.com/blog/ai/25/llm-inference-benchmarks-predibase-fireworks-vllm>
 91. Selecting GPUs for LLM serving on GKE | Google Cloud Blog, accessed April 15, 2026,
<https://cloud.google.com/blog/products/ai-machine-learning/selecting-gpus-for-llm-serving-on-gke>
 92. WANSPEC: Leveraging Global Compute Capacity for LLM Inference - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2602.18931v1>
 93. MIRA: Memory-Integrated Reinforcement Learning Agent with Limited LLM Guidance - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2602.17930v1>
 94. SkyServe: Serving AI Models across Regions and Clouds with Spot Instances - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2411.01438v2>
 95. 5 Things Developers Get Wrong About Inference Workload Monitoring : r/AI_Agents - Reddit, accessed April 15, 2026,
https://www.reddit.com/r/AI_Agents/comments/1rv66jf/5_things_developers_get_wrong_about_inference/
 96. LLM Inference Optimization — Prefill vs Decode | by Robi Kumar Tomar | Towards AI, accessed April 15, 2026,
<https://pub.towardsai.net/llm-inference-optimization-prefill-vs-decode-6e003d48b2ca>
 97. Monitoring vLLM Inference Servers: A Quick and Easy Guide, accessed April 15, 2026,
<https://www.dataunboxed.io/blog/monitoring-vllm-inference-servers-a-quick-and-easy-guide>
 98. Advanced Monitoring LLM inference and inference routing - IBM, accessed April 15, 2026,

- <https://www.ibm.com/docs/en/ai-optimizer-for-z/2.1.0?topic=routing-monitoring-llm-inference>
99. vLLM Metrics in Production - Andrey Krisanov, accessed April 15, 2026, <https://akrisanov.com/vllm-metrics/>
 100. Metrics - vLLM, accessed April 15, 2026, <https://docs.vllm.ai/en/stable/design/metrics/>
 101. Metrics That Matter for LLM Inference - Compute with Hivenet, accessed April 15, 2026, <https://compute.hivenet.com/post/llm-inference-metrics-ttft-tps>
 102. Autoscaling with KEDA — production-stack - vLLM, accessed April 15, 2026, https://docs.vllm.ai/projects/production-stack/en/latest/use_cases/autoscaling-keda.html
 103. KEDA Auto-Scaler for inference workloads | KAITO - GitHub Pages, accessed April 15, 2026, <https://kaito-project.github.io/kaito/docs/keda-autoscaler-inference/>
 104. Autoscaling Kubernetes workloads with KEDA using Amazon Managed Service for Prometheus metrics | AWS Cloud Operations Blog, accessed April 15, 2026, <https://aws.amazon.com/blogs/mt/autoscaling-kubernetes-workloads-with-keda-using-amazon-managed-service-for-prometheus-metrics/>
 105. Autoscaling vLLM with OpenShift AI model serving: Performance validation, accessed April 15, 2026, <https://developers.redhat.com/articles/2025/11/26/autoscaling-vllm-openshift-ai-model-serving>
 106. The Architect's Guide to LLM System Design: From Prompt to Production | by Vi Q. Ha, accessed April 15, 2026, <https://medium.com/@vi.ha.engr/the-architects-guide-to-llm-system-design-from-prompt-to-production-8be21ebac8bc>
 107. Efficient LLM Scheduling by Learning to Rank [Quick Review] - Liner, accessed April 15, 2026, <https://liner.com/review/efficient-llm-scheduling-by-learning-to-rank>